
PROFILING AND IMPROVING THE PYTORCH DATALOADER FOR HIGH-LATENCY STORAGE

*
A TECHNICAL REPORT

Ivan Svogor, Christian Eichenberger, Markus Spanring, Moritz Neun, Michael Kopp
Institute of Advanced Research in Artificial Intelligence
Untere Viaduktgasse 16, 1030 Vienna
Austria
`{name.lastname}@iarai.ac.at`

ABSTRACT

A growing number of Machine Learning Frameworks recently made Deep Learning accessible to a wider audience of engineers, scientists, and practitioners, by allowing straightforward use of complex neural network architectures and algorithms. However, since deep learning is rapidly evolving, not only through theoretical advancements but also with respect to hardware and software engineering, ML frameworks often **lose backward compatibility** and introduce technical debt that can lead to bottlenecks and sub-optimal resource utilization. Moreover, the focus is in most cases not on deep learning engineering, but rather on new models and theoretical advancements. In this work, however, we focus on engineering, more specifically on the **data loading pipeline** in the PyTorch Framework. We designed a series of benchmarks that outline performance issues of certain steps in the data loading process. Our findings show that for classification tasks that involve loading many files, like images, the training wall-time can be significantly improved. With our new, modified **ConcurrentDataLoader** we can reach improvements in GPU utilization and significantly reduce batch loading time, up to $12\times$. This allows for the use of the cloud-based, S3-like object storage for datasets, and have comparable training time as if datasets are stored on local drives.

Keywords Deep learning engineering · PyTorch Lightning · Data Loading · Performance Benchmarking

1 Introduction

Modern Machine Learning (ML) frameworks for deep learning (DL) come with a variety of out-of-the-box tools that help researchers and practitioners accelerate their work through a straightforward definition of neural network architectures, optimization methods, loss functions, logging, etc. By providing abstractions through multiple layers (e.g. storage, computing hardware, and even the neural network itself), ML frameworks allow end-users to focus on DL models, data reasoning and solving automation challenges. That focus on rapid model development often puts engineering behind ML frameworks to a secondary place [1]. For ML engineers, new frameworks raise concerns about potential bottlenecks, technical debt, and poor (but necessary) design choices, all of which can lead to inefficient training and solutions unfit for production environments.

With the continuous demand for testing new ideas and the increasing size of datasets [2], the performance of training the models is lately capturing more attention due to limited and expensive resources. Nowadays, for achieving competitive state-of-the-art performance in video, image and speech processing one needs to train models on petascale data [2], which does not only highlight the importance of the training performance, but also the necessity data loading performance. Therefore, profiling deep learning code is necessary as it may uncover bottlenecks coming from an inefficient model implementation, the ML framework in use, or even the hardware.

Broadly speaking, the training of any given DL model can be split into three main stages [1], **1) communication, 2) data-loading and 3) computation**. Popular ML frameworks provide various profiling tools for inspecting the performance

of a model for those stages. That usually entails resource related parameters, e.g. GPU/CPU utilization, function-call execution time, or power consumption. In this work, we use *throughput* as a convenient and a straightforward metric to evaluate training efficiency, since the above mentioned parameters can often be overwhelming and convoluted to interpret. With model throughput one gets an end-to-end performance of DL training, which can be defined in two different ways, a) processed data items per second, b) processed data size per second. Usually, when dealing with reading raw data items or copying data (to memory or GPU) a common unit is Mbit s^{-1} . However, for the training phase throughput is oftentimes expressed in img s^{-1} . Since this work is intended to shine a light on both aspects, both units are used to make the results easy to understand and comparable. With higher throughput, the training resource utilization should be increased, thus reducing the total training (wall) time. That said, in this report we focus on the data loading efficiency using PyTorch [3], and in particular on cloud based storage, containing clean ready-to-use datasets, accessible to anyone without the need (and possibility) to make local¹ copies.

More specifically, our engineering research question in this report is the following:

Keeping storage format, preprocessing (augmentation) and batch size fixed, how can we achieve high throughput under different latency scenarios (fast local storage and higher-latency remote storage) by optimizing the Python data loading code?

In the following sections, we address these questions by examining the data loading procedure in PyTorch. Furthermore, we show results of a variety of benchmarks that ultimately result in a new, more efficient dataloader, compatible to the existing one.

The contributions of this technical report are the following:

- We show that data loading throughput can be increased by the introduction of within-batch parallelism in the PyTorch Dataloader on a vanilla vision dataset and model, in particular in high-latency settings as when data is loaded from remote storage.
- We provide two drop-in replacements for the PyTorch Dataloader <https://github.com/iarai/concurrent-dataloader>.

1.1 Benchmarking setup and resources

To address the aforementioned questions, a series of experiments to benchmark the throughput are presented, through which we highlight bottlenecks during training a DL model. For that reason, it is important to have a systematic approach with a stable baseline. In profiling ML models, there are no standard metrics to evaluate training performance. Instead, it is common to use a widely known ML model and dataset, which we also do in this work.

Machine learning framework

We will be focusing on PyTorch [3], and investigate and compare the performance to PyTorch Lightning [4], which is a lightweight PyTorch wrapper for high-performance AI research. As a baseline for the conducted experiments we will use the ResNet-18 [5] architecture for benchmarking purposes, and train it on the ImageNet ILSVRC 2012 dataset [6]. Furthermore, by *default*, we refer to unmodified versions available on public repositories, for PyTorch² and PyTorch Lightning³.

Computing resources

As we are dealing with performance benchmarking, it is important to consider the computing hardware. All the hardware used in this work is shown in Table 1. The majority of experiments are performed on the node in Datacenter 1. However, to investigate the influence of networking and various file systems, we repeat some experiments in Datacenter 2, located in a different facility. Additionally, we also perform some experiments on Google Colab and AWS EC2 as a sanity-check (see Appendix, subsection A.1).

¹In this report, we refer to local storage as local drives of the training machine or a network file system, not RAM or GPU memory

²<https://github.com/pytorch/examples/blob/master/imagenet/main.py>

³https://github.com/Lightning-AI/lightning/blob/1.5.2/pl_examples/domain_templates/imagenet.py

Datacenter 1		Datacenter 2	
CPU	2x Intel(R) Xeon(R) Gold 6146 CPU @ 3.20 GHz	CPU	2x Intel(R) Xeon(R) Gold 6244 CPU @ 3.60 GHz
GPU	8x Tesla V100-PCIE-32GB (only 1 was used)	GPU	4x Tesla V100-SXM2-32GB (only 1 was used)
Storage	6x NVMe 7T, INTEL SSDPE2KE076T8	Storage	2x NVMe 11.7T, Micron_9300_MTFDHAL12T8TDR
RAM	754Gi (Samsung DDR4-2600)	RAM	754Gi (Hynix DDR4-2933)
Google Colab		AWS (p3.2xlarge) ⁴	
CPU	2x Intel(R) Xeon(R) CPU @ 2.30 GHz	vCPU	8
GPU	1x Tesla K80	GPU	1x Tesla V100
RAM	13 GB	RAM	61 GB
		Storage	SSD

Table 1: Computing platforms used for experiments

1.2 Measurements and calculations

Throughout this work we will be addressing three major metrics:

- runtime*, which represents the time difference ($t_f - t_i$), recorded as Unix timestamp, with (t_i) marking the beginning of the experiment and (t_f) the end. The beginning is considered as the time when the first batch is being loaded, and the ending is considered as the time when the training process finished.
- throughput* [img s^{-1}], which represents the number of loaded data items (in this case images) during the training process. To put it more formally, consider a training dataset of N items, $D = \{i_1, i_2, \dots, i_N\}$ and the number of epochs N_{epochs} . The image throughput is calculated as $T_{imgs} = (N_{epochs} \cdot N) / (t_f - t_i)$. We may assume that epochs have a similar duration as measurements confirm.
- throughput* [Mbit s^{-1}], represents the size of loaded data items during the training process. The throughput in Mbit s^{-1} is obtained as $T_{mbits} = \left(\sum_{n=1}^N \text{size}(item_n) / (t_f - t_i) / 1024^2 \right) \cdot 8$, where *size* is the function that returns the image size in bytes.

For the initial and final experiment, we also address the GPU memory and processing utilization, measured at 10 Hz, i.e. over a 100 ms timeframe⁵, in a sidcar process.

1.3 Data loading pipeline

Figure 1 illustrates the typical data loading pipeline with PyTorch and PyTorch Lightning. It highlights three important lanes, *Measured activities*, *Associated activities in ML training procedure*, *Generic ML data pipeline* and.

From a software perspective, if we consider the layer that performs the training as the top layer, and the layer that collects data the bottom layer, then the Generic ML data pipeline with PyTorch usually consists of three main classes, a) the ML model itself, b) the `DataLoader` and c) the `Dataset`. Looking at it from top-down, the ML model implements the model itself, its initialization, and the training loop, while it uses the `DataLoader` to trigger the batch loading process. The middle lane of Figure 1 follows the software layers by describing associated activities in the ML training procedure. We may notice that the `DataLoader` creates `Worker(s)`, that are class instances running as `Processes` in charge of creating batch index lists. Each index of the batch list represents an index associated to the training item (in our case, an image) that will be fetched using a `Fetcher` class instance. Furthermore, the `Dataset` implements the lowest level of the data loading process which fetches a single training item from the storage (regardless whether it is local or remote). In our case, that means loading a single image, and returning it to the `Fetch(er)`. From there, it is forwarded to the `Worker`, which then assembles a collected batch and returns it to the training loop. After the batch has been assembled, the data gets transferred to the training device and the training process can start. This process continues until all the items in the dataset are exhausted, and repeated for each epoch.

The left most lane, shows measurement points, i.e. parts of the code which we associate with specific and unique log entries so that we can extract the runtime, and visualize the execution order of main functions during the training phase. Therefore, the *Get batch* is associated with a batch log entry, and the function in charge for starting the batch loading process, *next_data* is responsible for triggering the data loading process. Considering the logged time, this includes the entire batch loading time, and the time to initialize this process (creating workers, fetchers, loading batch indexes,

⁵<https://developer.download.nvidia.com/compute/DCGM/docs/nvidia-smi-367.38.pdf>

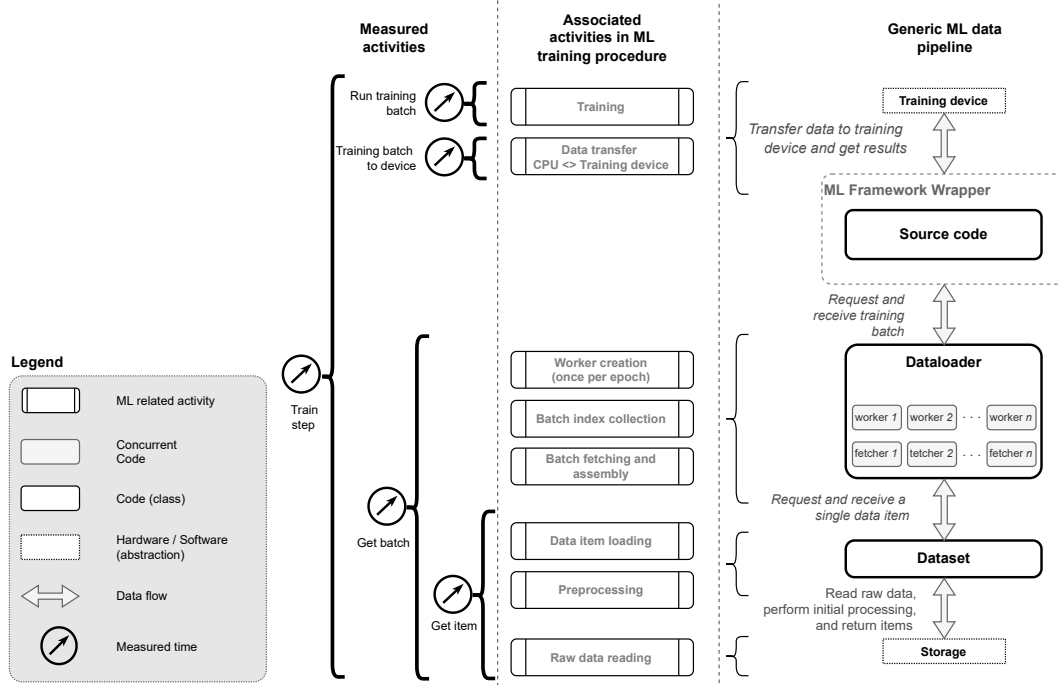


Figure 1: Simplified overview of the data loading pipeline in Torch/Lightning.

etc.), and loading a single item. Furthermore, *Get item* is associated with the log entry and function `__getitem__` implemented in the `Dataset` itself, and the logged time for it includes the time necessary to load the image from storage. If it is local, then one simply reads an image, however, if it is remote this time also include network-related overheads. Once the batch is returned to the main ML model class, using the log entry `training_batch_to_device` time to transfer the data to the GPU is logged, and finally, the time to run a training batch is logged.

1.4 Motivational experiment

In this section we introduce a motivational experiment to empirically verify whether and where we have bottlenecks in training using PyTorch. Let's consider the aforementioned Vanilla implementation of ResNet-18 in PyTorch and PyTorch Lightning, with the following parameters:

Batch size	Workers	Dataset limit (size)	Learning rate	Weight decay	Epochs
256	4	15000	0.1	0.0001	5

Table 2: Parameters for the motivational experiment

As mentioned, we use the ImageNet dataset, which consists of 14 million images, with average dimension of 469×387^6 . However, for the experiments presented in this work, we use a reduced set of images (`dataset_limit` parameter), and in the `Dataset` the following data augmentation (i.e. transform): 1) random resized crop to the dimension of 224×224 , 2) perform a horizontal flip, 3) convert to tensor, and 4) normalize.

The experiment was performed on the Datacenter 2 machine, using a single Nvidia Tesla V100 (SXM2-32GB) GPU as the training device, and we used *scratch* storage (locally mounted SSDs). The results of measurements are shown in Table 3.

⁶<https://towardsdatascience.com/compression-in-the-imagenet-dataset-34c56d14d463>

Storage	Lib.	$GPU_{util=0}$ [%]	$GPU_{util>0}$ [%]	$GPU_{util=0}^{mem}$ [%]	$GPU_{util>0}^{mem}$ [%]	Runtime [s]	Throughput [img s ⁻¹]	Throughput [Mbit s ⁻¹]
scratch	Torch	26.08	74.78	29.67	41.83	137.24	546.48	492.95
scratch	Lightning	66.41	64.68	5.79	18.93	491.06	152.73	137.77
s3	Torch	95.44	71.75	1.77	41.06	2309.99	32.47	29.31
s3	Lightning	98.04	65.28	0.34	19.19	8934.94	8.39	7.58

Table 3: Initial benchmark results, a comparison between PyTorch and Lightning using local storage (scratch), and remote storage (s3). Parameters according to Table 2.

In addition to the previously explained runtime and throughput, the table also shows four columns associated with the GPU:

- $GPU_{util=0}$, the percentage of experiment runtime where the GPU is not used.
- $GPU_{util>0}$, average GPU utilization (when GPU is not idle).
- $GPU_{util=0}^{mem}$, the percentage of experiment runtime where the GPU memory is not used.
- $GPU_{util>0}^{mem}$, average GPU memory utilization (when GPU is not idle).

For this experiment the GPU utilization is reported at a rate of 10 Hz, i.e. by averaging the utilization over a period of 100 ms. The experiment measuring the runtime and GPU utilization for the Torch implementation reported an average utilization of 74.78 %. However, 26.08 % of the overall experiment runtime the GPU was idle. Furthermore, GPU memory is not used 29.67 % of the experiment and is utilized on average with 41.83 %. For Lightning, the GPU idle time seems much larger, as well as GPU memory utilization lower. This indicates that there are significant overheads compared to the pure PyTorch implementation and there is room for improvement and fine-tuning since Lightning uses PyTorch "under the hood".

This experiment shows that there is considerable room for improvement if one is able to reduce the time in which the GPU is idle, namely 26 % for PyTorch and 66 % for Lightning.

Furthermore, when using AWS S3 remote storage the GPU idle time, and with it the runtime of the entire experiment, increased due to network latency. Therefore, when interpreting the GPU utilization in Table 3 with respect to Figure 1, one may conclude that a large portion of time is used on data handling and not for training. An attempt to make this conjecture visible is shown in Figure 2 which shows the timeline of the first 250 s of training. The horizontal lines in red, magenta and blue show the duration of the respective function calls in Figure 1 for loading a batch (*Get batch*), moving the data to the GPU (*Training batch to device*) and training on it (*Run training batch*). The cyan and brown dashed lines correspond to GPU utilization.

The right plot in Figure 2 shows that the *training* starts by downloading 4 batches in parallel (red lines). Once complete, there is almost an indistinguishable magenta dot that represents copying the training batches to the GPU. Finally, there is a short blue line that represents the training. One may notice that simultaneously with the start of the magenta line the GPU utilization and memory increases. Worth mentioning is that the first training loop is slightly longer than the remaining training iterations due to the initial resource initialization. In summary, this shows how the majority of time is used for data loading.

Figure 2 highlights several important considerations:

1. Most of the plot consists of red lines which represent the lifetime of a function in charge for loading the batch (*Get batch*).
2. We see discrete steps and GPU utilization peaks with long pauses in between.

This indicates that it takes only a short period of time to process data on the GPU for training compared to the time that is needed to load it from remote storage. For both Torch and Lightning, loading a batch from a remote storage takes more than 30 s. Left two plots of the Figure 2 show details for scratch and remote storage, with both Torch and Lightning. While for local storage, the batch loading time still takes the most time (between 1.68 s and 1.57 s (median)), comparing to S3 its only a fraction of time. As we do not have unlimited memory and since we cannot have an unlimited number of concurrent connections, we try to exploit the available network bandwidth by introducing within-batch parallel download of samples.

In the next section, we introduce an additional layer of parallelization and provide new experiments to explore how it affects the end-to-end performance.

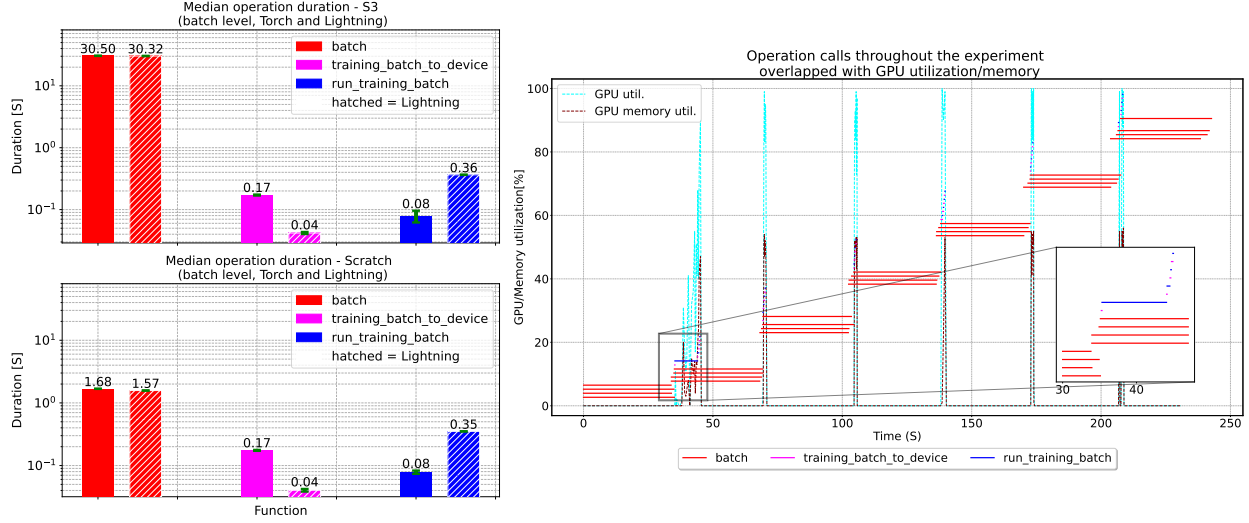


Figure 2: Graphical representation of data loading function calls; left image shows average time necessary to load a batch, transfer to a GPU and train, while the right image shows a timeline of function calls, and GPU utilization (zoomed in on the first 250 s of the experiment, with AWS S3 storage).

2 Data loading modifications

2.1 Problem analysis: data loading in PyTorch

While the data loading pipeline shown in Figure 1 uses parallelization over batches by utilizing multiple workers, the data items themselves are accessed sequentially within a batch⁷. To visualize, Figure 3 depicts the loading of a single batch which proceeds as follows. The `Dataloader`⁸ uses a `worker_loop`⁹ that gets attached to a new `Process`. Also, the `Dataloader` creates an index queue for each worker and populates it with a tuple consisting of the batch ID and indices¹⁰. Those indices correspond to data items to be loaded. To give an example, an entry $(3, [12, 13, 14, 15])$ would mean that the worker is loading batch number 3, consisting of four items (12...15). The worker then creates a `fetcher`¹¹, which takes the list of indices to fetch and proceeds to do so sequentially with the `__getitem__` function of the `Dataset`. Therefore, by using 4 workers at most 4 batches, are loaded in parallel, while the individual data items within a batch are loaded sequentially. This raises the question: why are individual data items fetched sequentially?

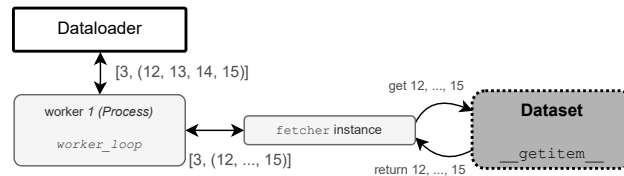


Figure 3: Simplified illustration showing sequential loading of data items.

2.2 Modifications: fetcher parallelization layer

In order to fetch individual data items in parallel, we propose adding an additional layer of concurrency, such that each worker can use multiple fetchers to get individual data items in parallel. Considering the previous example, this would correspond to fetching the items $[12, \dots, 15]$ in parallel.

We equip `_MapDatasetFetcher` (from `torch.data._utils`) with two new classes, each one using a different approach to parallelized downloads: `Thread Pool` and `Asyncio`. Threading is usually associated with parallelism using shared

⁷https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/_utils/fetch.py#L26

⁸<https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/dataloader.py#L904>

⁹https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/_utils/worker.py

¹⁰<https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/dataloader.py#L1220>

¹¹https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/_utils/worker.py#L268

memory. However, since Python uses the Global Interpreter Lock (GIL) to protect the internal state of the interpreter, all threads in Python are pinned to a single CPU, i.e. Process [7]. The benefit of threading is in preemptive switching automatically performed by a thread manager. However, this comes at a price of uncertainty since this switch may occur at any moment. Therefore, to prevent incoherent states, it is required to use critical sections (enabled by locks), which can be expensive when using a large number of threads¹².

On the other hand, Asyncio allows for concurrency within a single thread. By using the keywords `yield` and `await` task switching is performed manually. This minimizes CPU utilization and has fewer overheads than threads. The synchronization points are therefore no longer necessary. However, this requires a non-blocking version, or async version, of operations such as file reading, networking, etc.¹³.

Since both approaches have advantages and disadvantages with respect to this particular use case, the decision is to implement the existing `_MapDatasetFetcher` with both libraries.

- `_AsyncMapDatasetFetcher` is implemented using the Asyncio library. In this approach, `asyncio.Queue` is used for all items in the aforementioned batch index queue, and for each one creates a fetch task. Once this is completed, all tasks for fetching individual data items are started, and run asynchronously. When all the items are loaded, the batch is assembled and returned to the Worker, i.e. `DataLoader`. This implementation is referred to as *Asyncio implementation* in the text.
- `_ThreadedMapDatasetFetcher` has the same functionality as `_AsyncMapDatasetFetcher` but uses the `Threading` library instead. Furthermore, it also introduces an additional layer of parallelism in the worker which is explained in the following paragraph. This implementation is referred to as *Threaded implementation* in the text.

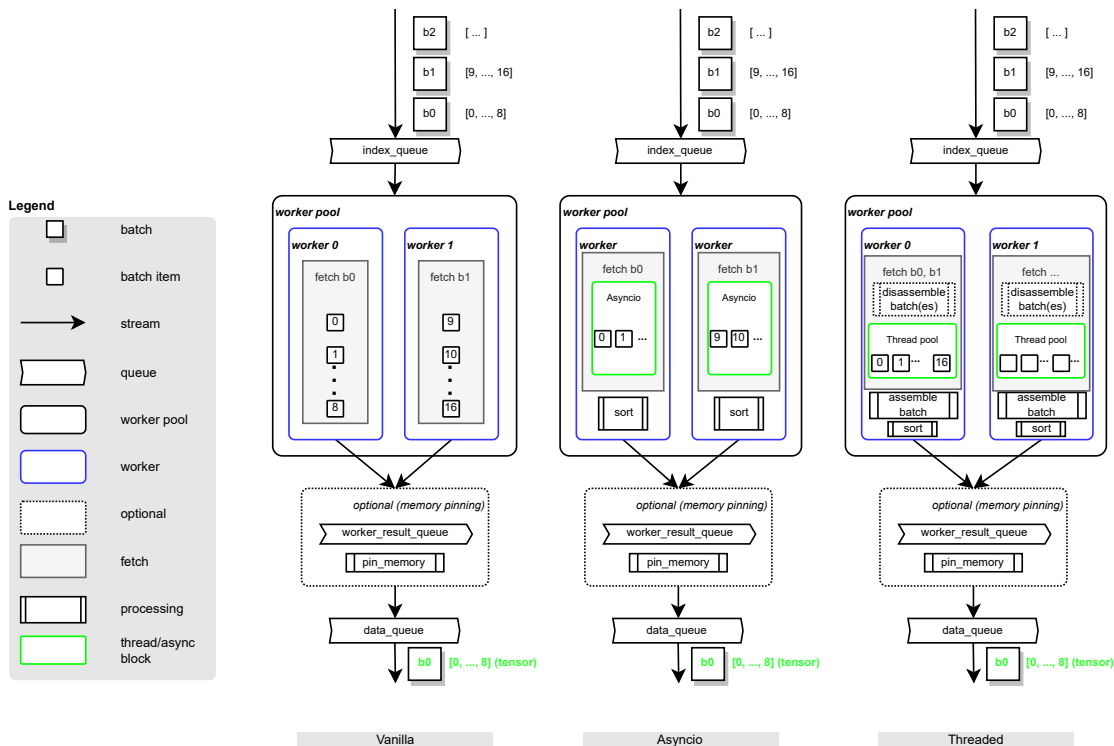


Figure 4: Implementation differences between the vanilla batch loading (left), and two new implementations introduced in this work; async and threaded (respectively),

To highlight the differences between the different implementations, consider Figure 4. In the illustrated examples two workers are used for loading the data items. The left-most part of the image shows the vanilla implementation of the batch loading in PyTorch. It allows for batch level parallelism, i.e. each worker processes a single batch at a time. The `number_of_workers` and the `prefetch_factor` determine the maximum number of batches loaded to memory and

¹²The authors are aware that the term concurrency is associated with Asyncio, while parallelism is associated with Threads. In this work these terms are used interchangeably.

¹³https://pybay.com/site_media/slides/raymond2017-keynote/intro.html

ready to be used for training. The items of the batch 0 (b0) are loaded into the `index_queue`, which is consumed by a worker (e.g. worker 0), which sequentially loads data items indexed 0, ...8. Optionally, if memory pinning is used the items are transferred to the pinned memory. At this point the batch is ready for the trainer to consume.

The middle part of Figure 4 illustrates the Asyncio implementation. At first, a list of indices for the individual data items within a batch are passed from the worker to the fetcher. However, instead of sequentially loading each data element, an Asyncio asynchronous block, which allows for concurrent execution, is added. Since there is again no guarantee that the data items are loaded in the requested order, the elements are sorted and put to the `data_queue` after the batch is complete.

Last but not least, the Threaded implementation is illustrated in the right part of Figure 4. It has one additional feature, which we tried out only in this implementation. In addition to the within-batch parallelism of Asyncio, we tried out batch disassembly in this implementation as well. Instead of loading a single batch the worker disassembles several batches into individual data items and proceeds to download them. The number of disassembled batches is controlled by the `batch_pool` parameter. If it is set to a value larger than 0 batches are disassembled, otherwise no batch pool is used. The use-case illustrated here, has `batch_pool` set to 16 which leads to two batches disassembled. After disassembling, a Thread pool is used to fetch all 16 items in parallel. As the data items are loaded, the batches are reassembled and their individual items are sorted to restore the requested order¹⁴. Finally, the batches are put into the `data_queue` to be ready for the trainer to consume them.

Both Asyncio and Threaded are intuitive to use, and almost invisible to the user. While constructing the `DataLoader`, users can specify two new parameters, along with the implementation choice; *Threaded*, *Asyncio* and *Vanilla*:

- *Number of fetch workers*, refers to the maximum number of threads the `ThreadPoolExecutor` can use to execute fetch tasks asynchronously.
- *Batch pool size*, referring to the aforementioned batch pool, i.e. the number of items that each worker can take from the given batches.

Table 4 summarizes the dataloading features and their mapping to the technical parameters.

Feature	Description	vanilla	asyncio	threaded
parallelism over batches	The number of batches that can be downloaded concurrently	<i>num_workers</i>	<i>num_workers</i>	$num_workers \times \frac{batch_pool}{batch_size}$
Batch queue size	<i>Backpressure</i> , i.e. the number of batches that needs to be loaded for training before a new batch is fetched	$num_workers \times prefetch_factor$	$num_workers \times prefetch_factor$	$num_workers \times prefetch_factor$
Batch item parallelism	The number of concurrent tasks that load single data items.	– (semantically 1)	<i>num_fetch_workers</i>	<i>num_fetch_workers</i>
Batch disassembly	The number of data items loaded concurrently, across multiple batches (as batches are disassembled).	– (semantically 1)	– (semantically 1)	<i>batch_pool</i>

Table 4: Feature to technical parameter mapping

2.3 Results

For the proposed modifications, we perform a benchmark to compare how do Asyncio and Threaded implementation perform against the Vanilla implementation. We reuse the same parameters as for the motivational experiment (Table 2), with several additions shown in Table 5.

Batch size	Workers	Prefetch factor	Number of fetchers	Batch pool	Dataset limit (size)	Learning rate	Weight decay	Epochs
256	4	4	16	0	15000	0.1	0.0001	5

Table 5: Parameters parallelization benchmarking

The experiment results are shown in Figure 5. The left side of the figure shows the results for the cloud based storage (S3), while the right part shows the results for local storage. The throughput for loading the data from S3 storage using the Asyncio and Threaded implementation compared to the Vanilla implementation improved by 11.44 and 10.77 $\times$, respectively, when used for the training setup with pure PyTorch. For the training setup with PyTorch Lightning the improvements are 32.94 and 39.20 $\times$ respectively. When loading the data from Scratch storage the gain in throughput for both, the Asyncio and Threading implementation, is 1.55 $\times$ for the training setup with pure PyTorch and 4.07 $\times$ for the PyTorch Lightning setup.

¹⁴ due to parallelism, there is no guarantee elements are downloaded in a specific order

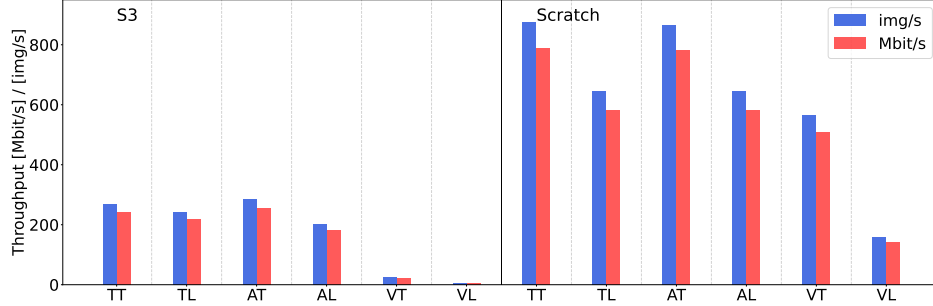


Figure 5: Benchmarking data loading without and with our modifications for s3 and scratch. The y -axis shows the throughput (Mbit s^{-1} and img s^{-1}) for the newly added parallelism in the fetching layer. The x -axis shows abbreviations for libraries and implementations (e.g. *TT* is PyTorch Threaded, *TL* is PyTorch Lightning, *AT* is Asyncio Threaded, etc.). Parameters according to Table 5.

The results show that cloud based storage is benefiting the most from the introduced layer of parallelism. This is expected, as there is substantial latency when fetching from cloud storage (see subsection 3.2), which is particularly bad under sequential access pattern. This implicates that additional fine tuning should produce even better results, which is a pointer for further investigation. Even with this change, local storage is outperformed in the case Vanilla Lightning vs any of our modifications.

Since Threaded implementation can come in two different forms, as explained previously, with and without the feature of batch disassembly (Figure 4), we performed an additional benchmark comparing the three different approaches, with the results shown in Figure 6.

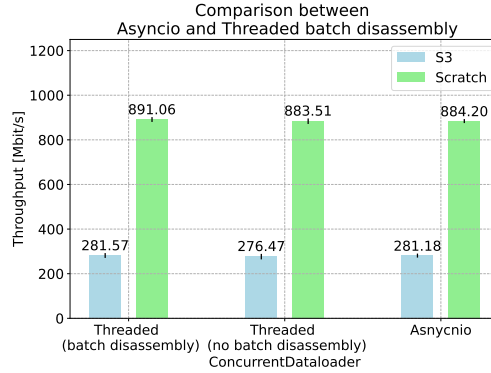


Figure 6: Comparison between different approaches of data-item parallelism. Threaded with and without batch disassembly vs. Asyncio.

The proposed feature, which disassembles a batch and then downloads data items from multiple batches in parallel, within a single worker seems to provide no significant improvement, for the proposed use-case. Henceforth, this feature will not be used.

2.4 Additional considerations: Process initialization and caching

Through various benchmarks performed for this work, we learned about two additional points that should be taken into consideration for a potential performance increase, `Process` creation and `DataLoader` initialization. Also, we look into the effect of using a web cache.

Process creation

Process creation, to initialize worker `Process(es)`, both PyTorch Lightning and PyTorch use Python’s `Multiprocessing` library. However, for PyTorch the default setting is `fork`, whereas for PyTorch Lightning it is the `spawn` method¹⁵.

¹⁵<https://docs.python.org/3.10/library/multiprocessing.html#contexts-and-start-methods>

The main difference between these two methods is that `fork` creates a child process, which inherits all resources from the parent, and that `spawn` creates a clean, new Python interpreter process, which takes significantly more time. This also means that when using the `fork` method, CPU and GPU calls cannot be mixed, which in PyTorch and PyTorch Lightning means that *memory pinning* cannot be used. This is due to the fact that memory pinning uses GPU calls to automatically load fetched data into page-locked (i.e. pinned) memory, resulting in faster host to GPU copying¹⁶.

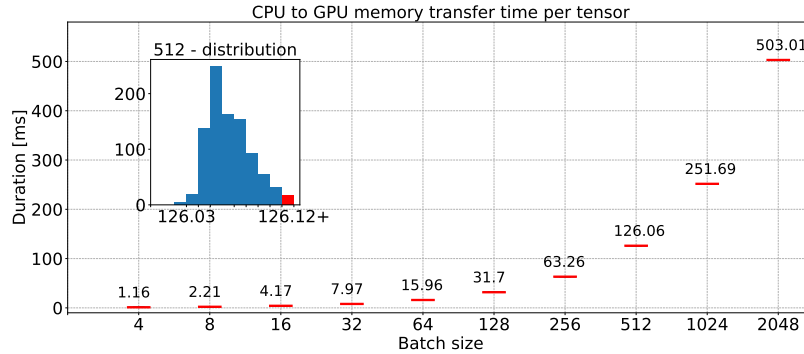


Figure 7: Increasing the batch size, increases the average CPU to GPU tensor transfer time. The inset plot shows the distribution histogram of transfer time for batch size 512, with the red bar representing the overflow bin that contains all outliers

Figure 7 shows that larger batch size significantly increase the CPU to GPU transfer time, so memory pinning should be an important feature. However, for our use-case the GPU transfer time (training batch to device, shown in Figure 2) is not significant comparing to the batch loading time, which we are trying to reduce.

Dataloader **initialization**

Dataloader initialization (constructor), creates `Process(es)` which are used as workers to load data items of a single batch. The issue here is that creating `Process(es)` can be slow which further delays the object creation¹⁷ and execution. It is not good practice to have bloated constructors since their role should be to construct objects. However, often it is also used for initialization or obtaining the necessary dependencies. In the case of the `Dataloader` constructor a red flag is the `Process` initialization, which may take considerable time and therefore block further execution. Suppose that the `Dataloader` is constructed with 16 workers (i.e. `Process(es)`) and each one is taking a second to initialize. This will take 16 seconds before the first process starts loading batches. Though there can be multiple solutions to deal with this, we introduced a modification that allows for lazy-loading and non-blocking process creation which is shown in Figure 8.

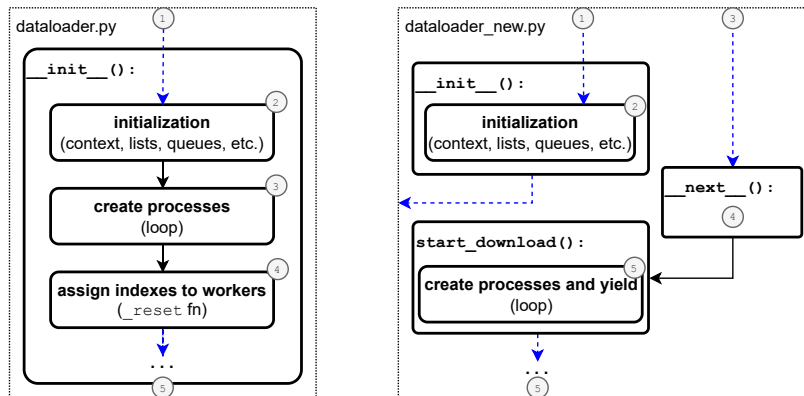


Figure 8: Simplified illustration of steps necessary to start the data loading. The left-hand side figure shows the original, while the right side shows a modified, non-blocking version with lazy initialization.

¹⁶<https://pytorch.org/docs/stable/data.html#memory-pinning>

¹⁷<https://github.com/pytorch/pytorch/blob/v1.9.1/torch/utils/data/dataloader.py#L904>

Blue dashed lines represent external calls coming to `dataloader.py`. Normally, as shown in the left side of Figure 8, (1) a call comes in requesting the initialization of the `_MultiProcessingDataLoaderIter`, an object in charge for loading data. Initialization takes place by asserting worker sizes (2), creating a multiprocessing context, initializing local counters, flags, lists, etc. Then (3), the code enters a process creation loop, which starts multiple new `Process(es)` (this is a parameter, selected by the user). The loop sequentially creates new processes. After completion, a `_reset` function (5) is called, which among other things, calls the `_try_put_index` function that is in charge for passing the index queues to individual workers.

We addressed the problem of the blocking loop by first utilizing lazy loading, and yielding the `Process(es)` that have already been created thereafter. Similarly then described above, (1) the multiprocessing context, local flags, lists, etc. are initialized. After this (2), the `_init__` function returns and allows for the `_MultiProcessingDataLoaderIter` object to be created. Thereafter, once the training loop requests a new batch (3), it uses the `__next__` function, which is the part of `_MultiProcessingDataLoaderIter` object to obtain the batch. At this point, a new function (`start_download`) is triggered (4), which creates new `Process(es)` (i.e. workers) within a loop (5). Instead of blocking, it yields the created `Process` and lets it proceed with the data loading, but not before the call of the new `_try_put_index` function which loads the necessary indexes, for the processes that have been created up to that point. This procedure is triggered for each epoch.

Caching

During each epoch, the training loop iterates through all the elements in a dataset. When considering cloud storage, this means downloading the same data over and over again. This download uses HTTP requests to fetch individual data items which opens the question of caching. Caching would allow to keep local copies of items that have already been downloaded and reuse them. Also, by using caching libraries we can determine how many items we want to store locally.

For this purpose we use Varnish¹⁸, which is usually used as an HTTP accelerator for heavily consumed API endpoints. It intercepts HTTP requests at the OS level and checks if the data is already available locally. If this is the case (cache hit), it returns the cached item and otherwise (cache miss) downloads it.

Using the same parameters as for the previous experiment (shown in Table 5), Figure 9 shows the benchmark results with and without caching. To restrict caching to the use case that is studied in this work, which is that local storage is not sufficient to hold the full dataset, the caching size is set to 2 GB.

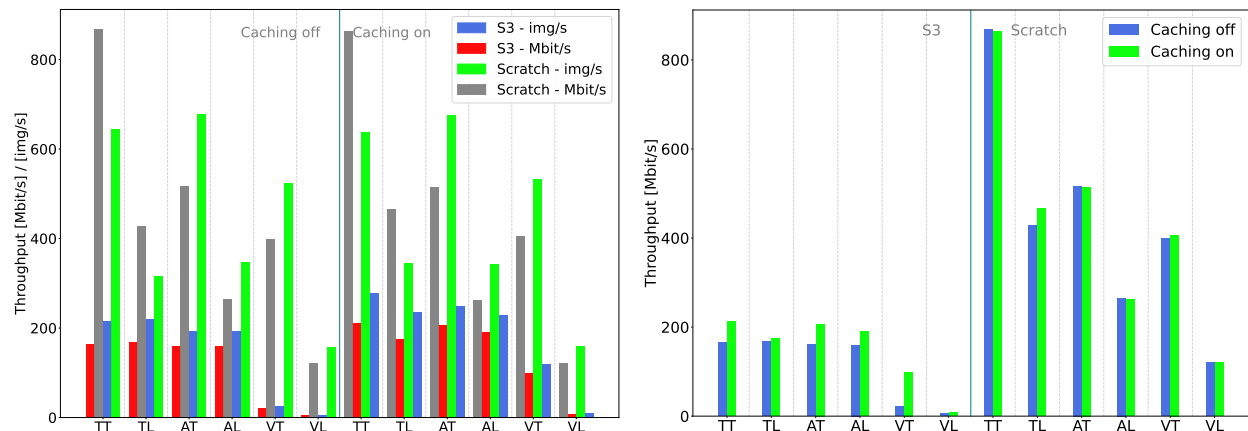


Figure 9: The left figure shows the end-to-end throughput for both images and $Mbit\ s^{-1}$, with and without caching for both scratch and S3 storage. The x -axis shows abbreviations for libraries and implementations (e.g. TT is Torch Threaded, TL is Torch Lightning, AT is Asyncio Threaded, etc.). The figure on the right highlights throughput differences in $Mbit\ s^{-1}$ for S3 and scratch.

For S3, the most significant improvements were found for Threaded Torch (28 %) and Vanilla Torch (450 %), and no improvements for the others (e.g. Threaded Lightning, Vanilla Lightning). However, these results need to be taken with a grain of salt. Since the caching size is limited the cache lookup often results in a cache miss, which requires the requested item to be downloaded again. This is due to the fact that during each training iteration the access pattern to

¹⁸<https://www.varnish-software.com/wiki/>

the items is random. As a sanity check we also use caching for local storage and found that there are no significant differences as one would expect. This leaves cache as an open question and a choice, depending on a particular use-case.

3 Concurrency parameters for the Dataset and the DataLoader

We demonstrated significant improvements in training runtime and throughput by introducing a new layer of concurrency. Furthermore, we improved the `DataLoader` initialization process and addressed caching. However, there is still a gap in performance between S3 and local storage that may be investigated further. In this section, we study the performance gap between PyTorch and PyTorch Lightning, which is substantial. In addition, we address how to select the concurrency parameters to squeeze the most of our hardware. So far, we have observed that a large portion of the experiment runtime is used for data loading and not for actual training. Therefore, we focus on the performance of the `DataLoader` (*Get batch* in Figure 1), and the `Dataset` (*Get item* in Figure 1) in the following experiments.

3.1 DataLoader

By introducing a new concurrency layer, new parameters have been introduced. However, the most important and influential ones are the number of fetch workers in combination with the number of workers. Those two parameters define how many parallel Processes or Threads are used. This can have a major impact on possible bottlenecks that may arise from resource locking. The effect of these two parameters on the throughput is measured with the following experiment setting.

Batch size	Number of batches	Storage	Number of workers	Number of fetchers
64	40	S3, scratch	1, 2, 4, 8, 16, 32, 64, 128	1, 2, 4, 8, 16, 32

Table 6: DataLoader benchmark parameters

Figure 10 shows how increasing the number of fetchers and workers using S3 storage and Threaded implementation, influences the overall throughput in Mbit s^{-1} (left), as well as the median request time in s (right). Considering the left heatmap, we may notice that the best performance is achieved when 1 – 4 fetchers and 32 – 128 workers are used. It also shows that a high number of workers and fetchers, as well as low numbers, result in suboptimal performance. In the right heatmap, we may observe that the median request time is highest when we have many workers and fetchers, which may be related to resource locking.

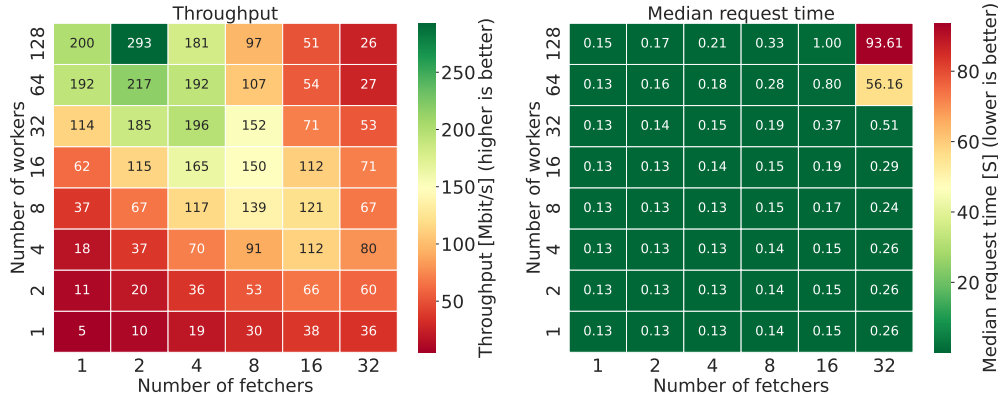


Figure 10: Heatmap plot, showing how the number of workers and fetchers with S3 storage influence the throughput (left), and the response time (right), using the Threaded implementation.

Figure 11 shows the same analysis but for scratch storage. Besides the much higher throughput, we may notice that for median request time (right plot), we have similar deterioration in performance for a large number of workers and fetchers. However, the throughput seems to be more stable with respect to the number of fetchers compared to S3 storage. As a result, choosing the number of workers higher than 16 leads to decent results, with the best throughput being achieved for 2 – 8 fetchers. This leads to the conclusion that the benefit from fetching is much less pronounced, since the median request time is already low and the networking overhead of remote storage is not present.

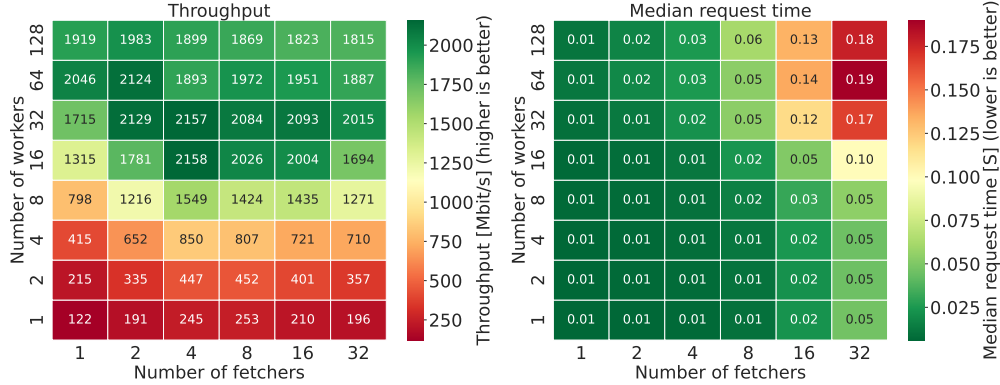


Figure 11: Heatmap plot, showing how the number of workers and fetchers with scratch storage influence the throughput (left), and the response time (right), using the Threaded implementation (throughput values are rounded to the nearest decimal)

3.2 Dataset

A layer below the Dataloader is the Dataset (Figure 1) which directly loads the images from storage. At this level, there are no workers or fetchers. Therefore, it can be instantiated separately, since it is almost completely isolated from the ML framework. Considering that this layer is accessing individual data items, it is important to understand how concurrency influences its performance. In this section, we address the concurrency in the context of pure image loading using the Dataset instance and by increasing the multiprocessing pool size. As the Dataset usually performs data augmentation, we did not exclude it from this benchmark. For this purpose, after the image is loaded from storage we perform 1) random resized crop to the dimension of 224x224, 2) horizontal flip, 3) conversion to tensor, and 4) normalization. The parameters used for the experiment are the following:

Pool size	Random images loaded	Number of image groups (batches)	Storage
1, 2, 3, 4, 5, 6, 7, 10, 15, 20, 30, 40, 50, 60, 80	2000	40	S3, scratch

Table 7: Dataset benchmark parameters

In the experiment, 40 image batches (i.e. batches, but not to be confused with batches from the context of a Dataloader, as Dataloader is not used here) loaded using the pure Dataset object (without the upper layers of the data loading pipeline Figure 1), each accessing 2000 random images while increasing the multiprocessing pool, that refers to the maximum number of parallelized functions with Python’s `multiprocessing.Pool`¹⁹. To randomly load an image, we’ve added a `get_random_item` function that randomly generates an integer index of an image, and then uses the usual `__getitem__` function to fetch the image from S3 or scratch. This also means that the same data augmentation is performed, as in all other experiments.

Figure 12 shows experiment result, i.e. the throughput and response time, for the increasing multiprocessing pool size, for both S3 and scratch, repeated 10 times. On both plots, the blue lines represent the achieved throughput, for a certain multiprocessing pool size. For S3, we can notice that after 30 simultaneous processes, there is almost no improvement in the throughput, which peaks around 75 Mbit s^{-1} . This is very similar to the previous case with the Dataloader (Figure 10) where we had the highest throughput for 32 workers and above, but then each worker (i.e. a process) had additional parallelized fetchers, which here is not the case.

In this experiment, using only the Dataset, the concurrency from multiple fetchers and workers is not present, and the only concurrency is on the image loading level, i.e. accessing random images. In terms of the Dataloader this corresponds to having no workers but multiple fetchers (in this case, pool size). The maximum we are getting here, represents a maximum we can get per parallelized fetcher, i.e. roughly 75 Mbit s^{-1} . However, the Dataloader also uses Processes to represent workers, so, for instance with 4 workers, we could technically get 300 Mbit s^{-1} .

The red dashed lines show the request time which is the time necessary to read and return a single data item (with data augmentation included). Given that values are ranging from 0.01 s up to 0.43 s, it is hard to conclude whether the

¹⁹<https://docs.python.org/3/library/multiprocessing.html>

request time depends on networking parameters, number of threads, or both. However, the median value suggests that between 20 and 60 workers in the pool, the response time is the highest. Furthermore, comparing to measurement with respect to Scratch storage on the right, it does suggest that networking introduces unpredictable behavior with respect to the request time. This might be the result of network latency, load, hardware, routing, etc.

For scratch, the results are much different, showing two distinct throughput peaks, one for 2 processes around 310 Mbit s^{-1} , and another between 15 and 20 processes giving between 210 and 250 Mbit s^{-1} . As for the response time, unlike with S3, for scratch, it is much more predictable. Between 2 and 20 processes, peaking 10 we have the largest response times, and considering the throughput, 2, 15 or 20 processes are the best choice since then we have the highest throughput and lowest response time.

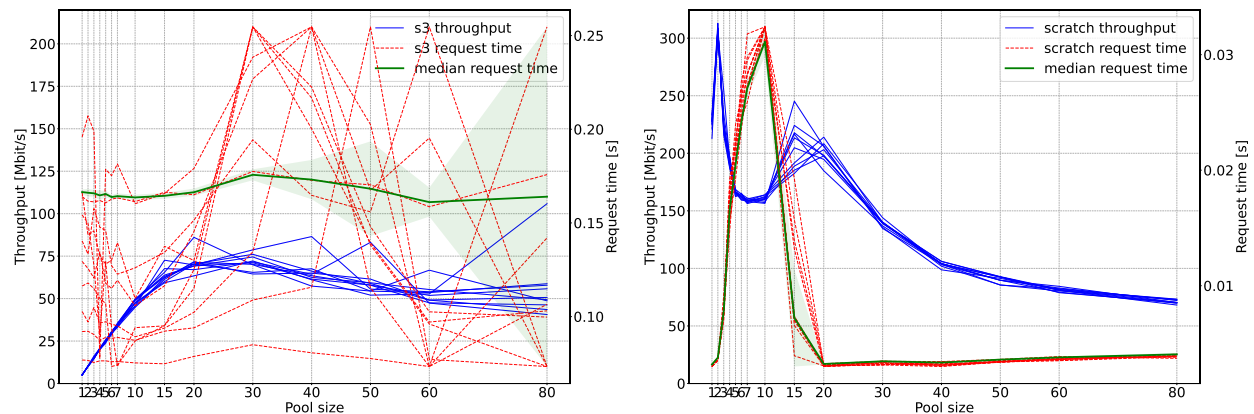


Figure 12: Throughput (left y -axis) and request time (right y -axis) for random image loading using a Dataset object, for different multiprocessing pool sizes, for S3 (left) and scratch (right). Points of the blue line, represent throughput over the entire experiment, while the request time represents a median response time.

This experiment highlights the main throughput and response time differences between using local and remote storage. For accessing remote storage, concurrency is key. Also, this experiment outlines what is the maximum performance we can get for image loading, using Python multiprocessing. Considering that the throughput from the Dataloader experiments are much higher, for both types of storage, a combination of multiprocessing, threading and asynchronous requests provides the best data loading performance.

4 Complete end-to-end benchmark

Each of the previous experiments addresses a certain aspect for dealing with remote storage and concurrency. The evaluated modifications include a concurrency layer in the fetcher, additional thread pool for the workers (only in Threaded implementation), lazy initialization of the Dataloader, and caching. Furthermore, the maximum throughput of each data loading layer has been explored. In this section, we present the initial experiment (see Table 3) repeated with all the aforementioned modifications.

Figure 13 shows the results, for all combinations of implementations and libraries, including the throughput and GPU utilization (processing and memory). The motivation for this work is to approach the throughput of local storage, and increase the GPU utilization, when loading data from a remote storage. Even though the introduced modifications do not outperform local storage, they lead to a substantial increase in terms of throughput. With the modifications introduced in the Threaded Torch implementation to load data from S3 it is possible to reach 67 % of the Vanilla Torch implementation with respect to loading data from scratch. This is a $15.5\times$ improvement compared to the Vanilla Torch implementation. With Lightning, it is possible to achieve similar performance. Even more, it is possible to outperform Lightning scratch with the Lightning Threaded implementation $2.5\times$. In addition to these results, Figure 13 indicates that the proposed modifications also help in improving the performance when using local storage.

The GPU processing and memory utilization, as depicted in Figure 13, shows that the GPU idle time is significantly reduced with our modifications, in particular when using remote storage. For GPU memory utilization there are only slight differences, which is expected given that the model itself, nor batch sizes were changed.

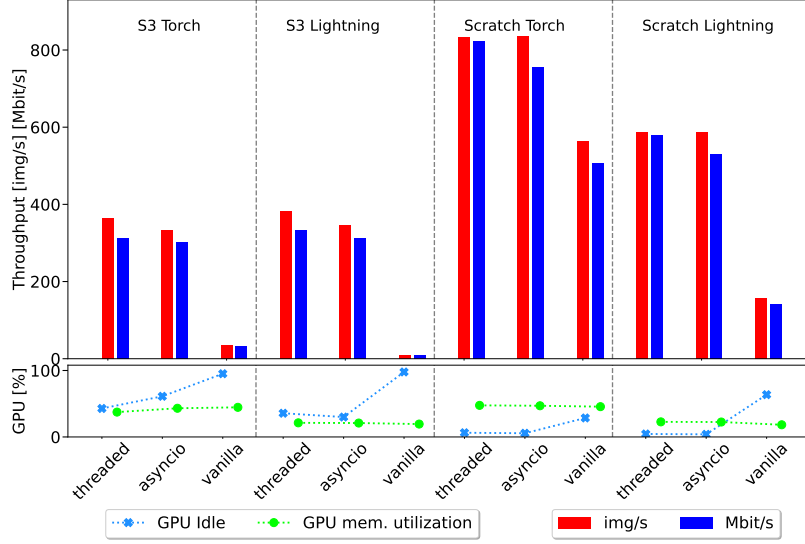


Figure 13: Initial experiment repeated, with all the introduced modifications, but with original parameters to keep the experiments consistent and comparable to the motivational experiment. S3 vs Scratch, with Threaded, Asyncio and Vanilla implementation, including the GPU idling and memory utilization.

Figure 14 shows the median duration of the main functions used for training. The figure highlights significant improvements in batch loading. This improvements lead to a reduction of batch loading time of up to 12 $\times$ for S3 cloud storage, and up to 3 $\times$ for Scratch storage.

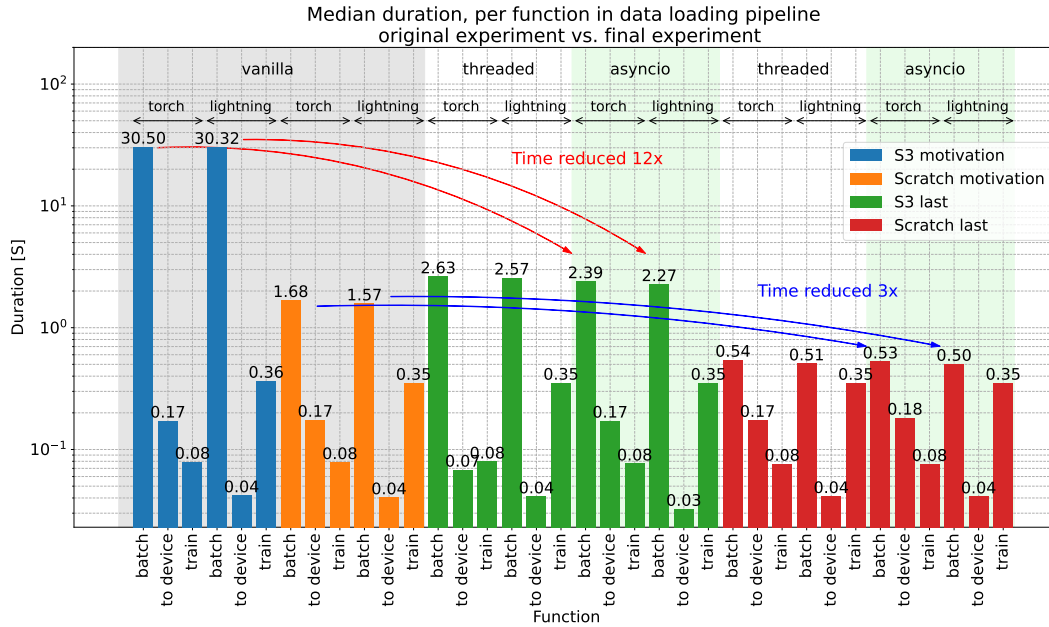


Figure 14: The figure highlights significant improvements for both S3 and Scratch storage that can be achieved with modifications to the data loading pipeline introduced in this work.

4.1 End-to-end throughput per data loading layer

With the variety of presented experiments we are able to compose a figure that shows throughput rates per layer in the data loading pipeline. Figure 15 is the extension of Figure 1 that clearly shows how it is possible to benefit from concurrency. Starting with the pure Dataset with concurrency at the bottom, we can expect throughput between 4

and 79 Mbit s^{-1} for S3, and between 73 and 304 Mbit s^{-1} for scratch. In the next layer up, with the `Dataloader`, that combines threading and multiprocessing, this significantly increases, so we can get between 5 and 293 Mbit s^{-1} for S3 and between 121 and 2159 Mbit s^{-1} for scratch.

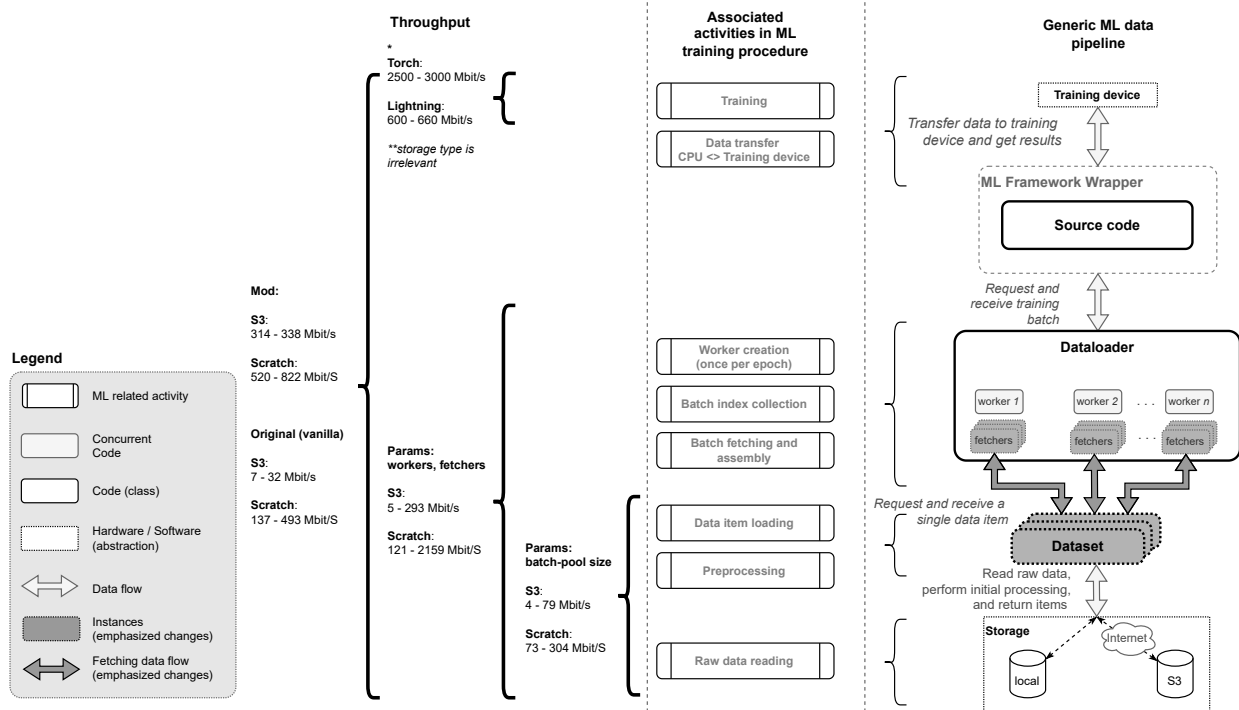


Figure 15: The image highlights the changes and the throughput the data loading pipeline, along with the throughput of the training phase (* which is further addressed in the appendix) Values shown here show the throughput range which depend on different implementations (e.g. Threaded vs Asyncio, and Threaded vs Lightning).

However, for the end-to-end which includes the entire training process this gap closes down a bit, given that for S3 we get throughput between 314 and 338 Mbit s^{-1} , and for scratch between 520 and 822 Mbit s^{-1} . Note, that for scratch, we also included numbers that use our modified data loading stack. Compared to the original numbers, as already stated preciously, we can get up to 68.5 % of scratch throughput performance.

Given the numbers presented above, one can notice that while the highest throughput for scratch storage is 2159 Mbit s^{-1} on the `Dataloader` layer, with end-to-end the highest throughput was 822 Mbit s^{-1} , which can indicate that the training bottleneck is no longer in the data loading pipeline. The details of the training step are addressed in depth in the appendix.

4.2 Related work

The related work on this subject is relatively scarce and published in a variety of venues. This indicates that ML engineering does not quite get the attention that it deserves. Nevertheless, more frequently journals and conferences call for special issues and tracks on ML engineering. The relevant publications can fit into two major categories, a) ones dealing with data and case specific datasets (e.g. Omnidata [8] and Kaolin [9] focusing on 3D data, TorchMeta [10] focusing on few-shot learning, TorchIO [11] focusing on medical datasets, etc.), b) those emphasizing importance of efficient data loading, with the latter being the most related to the present work.

Yang and Chong present an analytical model that helps analyze the cost of data loading in a distributed training environment [1]. The authors also mention and partially exploit the untapped parallelism for loading single batches, similarly as presented in this paper. However, the main focus of their work is aggregated caching, or *locality-aware caching*, that provides a way for individual nodes to train on the data that is already stored in the local cache. The authors report minimizing the overall data loading time and a $30\times$ speedup in data loading (with 256 nodes). Furthermore, the authors point out that in their case Python I/O operations were chosen such that they release GIL. Though in our work we do not address distributed training, Yang and Chong provide a roadmap for our work in that direction.

Another important issue, with data loading, is data augmentation which is the focus of work by Zolnouri et. al. [12]. The authors report that heavy data augmentation has a great impact on the standard PyTorch Dataloader, utilizing up to 40% of the training time in their use case. Using Nvidia’s DALI²⁰ they were able to improve the loading of data from Imagenet by a factor of 100. With DALI, the data loading process is shared between CPU and GPU, and therefore the augmenting operations can be run on both. This further confirms the issue with the PyTorch data loading pipeline.

Furthermore, in work by Aziman et.al. [2] from Nvidia, a great overview of data loading techniques is presented and argued how efficient data loading is essential for state-of-the-art deep learning. The authors present a new data loading pipeline called AI Store, which seeks to address the drawbacks of distributed existing filesystems. In their work the authors point out that these are not made for data access patterns used in DL. This includes iterating over a random permutation of a training dataset. Also, the authors call for a framework-agnostic data loading solution to optimize for DL requirements.

In this work we have shown empirically how the Torch DataLoader can benefit from additional layers of parallelism. The presented increase of the DataLoader efficiency due to the implemented modifications opens a gateway towards effective use of datasets stored on remote servers and object stores (e.g. AWS S3, Google Cloud, Azure Storage, etc.) to improve and simplify the dataset creation, management, and distribution. However, by using shading, FastAI and WebDataset can outperform our Dataloader (Appendix, subsection A.5). Whether someone wants or can use sharding is beyond this discussion, however for our future work it should be a consideration.

5 Conclusion and Future Work

Contribution This technical report shows that data loading throughput can be increased by the introduction of additional concurrency and some minor changes the PyTorch DataLoader, in particular in high-latency settings as when data is loaded from remote storage. Specifically, on a vanilla vision dataset and model, we increased the end-to-end throughput performance by up to $15.5\times$ for fetching from S3, corresponding to 67% of the vanilla PyTorch implementation from local SSD; batch loading time could be reduced by up to $12\times$ for S3 cloud storage, and up to $3\times$ for Scratch storage.

Limitations and Future work Such improvement opens up the possibility to replace local, i.e. on-site storage with remote cloud-based storage, and allow for: a) creating a centralized data storage registry that wouldn’t require making needless data copies, b) further decouple data and data loading from the ML framework.

In our solution, we did not consider additional operations on the computing platform or datasets (such as, caching, sharding, data unpacking, etc.). With such additional techniques the end-to-end throughput could be even better. That said, we are incentivized to further explore this work, and in particular compare it to similar platforms like DALI, Ray, Tensorflow, etc. To what measure do those platforms rely on on-disk staging, dataset modifications, dataset preprocessing on the remote-storage side, etc? To what degree can our cache-free and shard-free compete with existing solutions?

Furthermore, the implementation in a lower-level language should allow for more efficient access to networking resources, parallelization (see appendix, subsection A.4) and potentially low-level communication features between the CPU and GPU. Of course, a framework like this should be accessible easily through Python, as it became a de-facto domain-specific language of machine learning.

Code Availability

Our concurrent data loader is available under <https://github.com/iarai/concurrent-dataloader>.

Author Contributions Statement

Roles according to CRediT (Contributor Roles Taxonomy)²¹: I.S. software, investigation, visualization, writing (original draft, review & editing); Ch.E. software (early), writing (review & editing); M.S. writing (review & editing); M.N. validation, conceptualization; M.K. conceptualization.

²⁰<https://developer.nvidia.com/dali>, a portable, open-source library for decoding and augmenting images, videos and speech to accelerate deep learning applications

²¹<https://credit.niso.org/>

References

- [1] Chih-Chieh Yang and Guojing Cong. Accelerating data loading in deep neural network training. In *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pages 235–245, 2019.
- [2] Alex Aizman, Gavin Maltby, and Thomas Breuel. High performance i/o for large scale deep learning. *2019 IEEE International Conference on Big Data (Big Data)*, pages 5965–5967, 2019.
- [3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [4] William Falcon and The PyTorch Lightning team. PyTorch Lightning, 3 2019.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [7] C. Hattinigh. *Using Asyncio in Python: Understanding Python’s Asynchronous Programming Features*. O’Reilly Media, Incorporated, 2020.
- [8] Ainaz Eftekhari, Alexander Sax, Jitendra Malik, and Amir Zamir. Omnidata: A scalable pipeline for making multi-task mid-level vision datasets from 3d scans. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10786–10796, October 2021.
- [9] Krishna Murthy Jatavallabhula, Edward Smith, Jean-Francois Lafleche, Clement Fuji Tsang, Artem Rozantsev, Wenzheng Chen, Tommy Xiang, Rev Lebedev, and Sanja Fidler. Kaolin: A pytorch library for accelerating 3d deep learning research. *arXiv preprint arXiv:1911.05063*, 2019.
- [10] Tristan Deleu, Tobias Würfl, Mandana Samiei, Joseph Paul Cohen, and Yoshua Bengio. Torchmeta: A meta-learning library for pytorch. *arXiv preprint arXiv:1909.06576*, 2019.
- [11] Fernando Pérez-García, Rachel Sparks, and Sebastien Ourselin. Torchio: a python library for efficient loading, preprocessing, augmentation and patch-based sampling of medical images in deep learning. *Computer Methods and Programs in Biomedicine*, 208:106236, 2021.
- [12] Mahdi Zolnouri, Xinlin Li, and V. Nia. Importance of data loading pipeline in training deep neural networks. *ArXiv*, abs/2005.02130, 2020.
- [13] Jeremy Howard and Sylvain Gugger. Fastai: A layered api for deep learning. *Information*, 11(2), 2020.

A Appendix

A.1 Exhaustive benchmark of different storage

Figure 16 shows the average throughput, along with error bars for an exhaustive benchmark performed on different storage types. The parameters were the following:

Batch size	Workers	Prefetch factor	Number of fetchers	Batch pool	Dataset limit (size)	Learning rate	Weight decay	Epochs
64	4	2	16	512	35000	0.1	0.0001	100

Table 8: Parameters for exhaustive benchmarking

To compensate for the fade-in and fade-out effect (see subsection A.6), the experiments were set to run for a longer period than previous ones (100 epochs). For Ceph object store (Ceph OS) and Ceph file system (Ceph FS) we used the Datacenter 2, for the Gluster file system (Gluster FS) we used the Datacenter 1, and finally, for S3 we used AWS EC2 instance.

To get an idea about the measurement error, the experiments with Gluster FS and Ceph FS were repeated 10 times to, while the Ceph OS was repeated only 6 times, due to its long duration. For clarity, the Vanilla Lightning experiment with Ceph OS ran for 18 hours, and repeating it 6 times meant runtime of 4.5 days. Just for a single experiment. For similar reasons, the experiment with EC2 using S3 (object store), was not repeated, and thus, doesn't show the error bars.

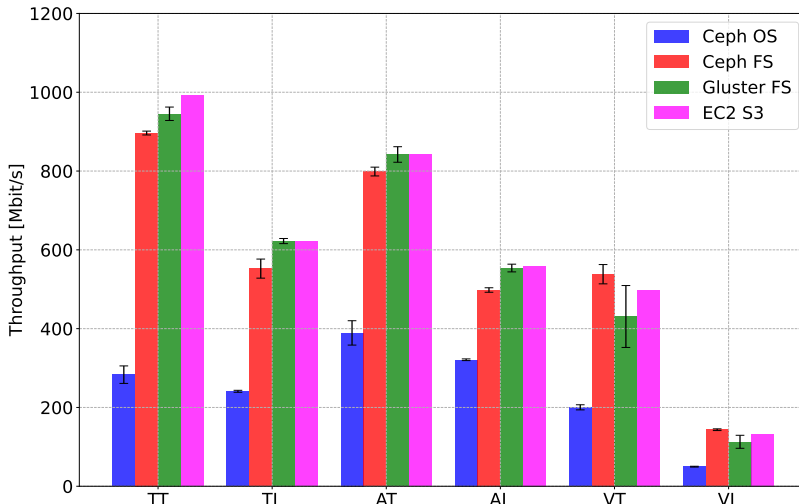


Figure 16: Throughput (Mbit s^{-1}) comparison between different storage types. Gluster FS (Datacenter 1) and Ceph FS (Datacenter 2) experiments are repeated 10 times, Ceph OS 6 times (due to its duration, on Datacenter 1), and S3 with EC2 machine, only once. The x -axis shows abbreviations for libraries and implementations (e.g. TT is Torch Threaded, TL is Torch Lightning, AT is Asyncio Torch, etc.)

For most storage types, that is, for Gluster FS, Ceph FS, and S3, the results are similar, however, for Ceph OS the throughput was significantly lower. Also, we can see that for the significantly longer benchmarks, compared to previous ones, the modifications in the data loading stack make a significant difference compared to the Vanilla version.

A.2 Sanity check: Google Colab

We ran a similar experiment on the Google Colab platform. Again, we used AWS S3, however, we weren't able to run multiple experiments due to several usage limits warnings: *GPUs and TPUs are sometimes prioritized for users who use Colab interactively rather than for long-running computations, or for users who have recently used less resources in Colab. As a result, users who use Colab for long-running computations, or users who have recently used more resources in Colab, are more likely to run into usage limits and have their access to GPUs and TPUs temporarily restricted.* That said, we reduced the number of epochs, and ran a single experiment with the following parameters (for pure Torch library, with Threaded, Asyncio, and Vanilla implementation):

Batch size	Workers	Prefetch factor	Number of fetchers	Batch pool	Dataset limit (size)	Learning rate	Weight decay	Epochs
64	4	2	16	256	3000	0.1	0.0001	5

Table 9: Parameters for Google Colab benchmark

We were able to only measure the experiment duration, however given that the average image size is 115 kB we were able to (roughly) infer the throughput from this. The results are shown in the table:

Implementation	Time [s]	Total images (dataset · epochs)	Throughput [img s ⁻¹] (total images/time)	Handled data size [Mbit] 1 img=115 kB=0.92 Mbit	Throughput [Mbit s ⁻¹] (datasize/time)
Asyncio	263.08	15000	57.02	13800	52.46
Threaded	264.16	15000	56.78	13800	52.24
Vanilla	385.93	15000	38.87	13800	35.76

Table 10: Experiment runtime on Google Colab platform shows that with the modified data loading stack, we were able to benefit from an extra layer of concurrency. The table also shows the inferred throughput.

A.3 Lightning vs Torch

A.3.1 Performance differences I: data loading

Throughput the many benchmarks performed in this work, we noticed lower performance from Lightning, than from Torch. Since Lightning is a wrapper that uses the Torch framework, it is interesting to report where the performance difference is coming from. We produced a new experiment with the following parameters to demonstrate the performance difference:

Batch size	Workers	Prefetch factor	Number of fetchers	Batch pool	Dataset limit (size)	Learning rate	Weight decay	Epochs
256	4	2	16	512	2048	0.1	0.0001	1

Table 11: Parameters for comparison in performance for PyTorch and Lightning

By adding additional logging, we were able to produce a complete picture of the execution order for each batch and epochs. It is displayed in Figure 17 below.

The image shows Lightning implementation with two epochs, where each one progresses through 8 batches. There are 4 workers, and the prefetch factor is 2. With our modifications, as soon as workers are created, each one gets a batch to download, so, initially we see 4 read lines where each one represents a worker collecting a batch. In this case, the prefetch factor 2 means that we need at least two batches before continuing to the training process. Now, let’s consider a highlighted batch in the Epoch 2:

- *Advance*, is the lane that indicates the run of the Lightning function call advance which uses a single batch to train. This is a function containing all the subsequent steps.
- *Prerun*, is a lane that is measured from entering the advanced function till the data is loaded to the device. In this case it encapsulates the following two lanes, next data and to device.
- *Next data*, is the lane that shows the execution of the function next, that triggers the loading of the next batch.
- *To device*, is the lane that shows the execution of the batch_to_device function which copies the batch to the GPU memory.
- *Get next batch*, it is a lane that shows the execution of functions involved in obtaining the training batch. It’s rather complicated under the hood, so the Figure 18 illustrates the procedure. Once the next function is triggered in the advance call, it triggers the `__next__` function (1) in the DataLoader. It then starts our function to `start_download` (2), that starts creating processes and triggered data fetching. However if it’s already downloading, it just returns (3). Then (4) the `_next_data` waits until the requested data arrives. When that happens function `_process_data` (5) is called. It triggers the download of the next batch, and returns the data of the current one (6). That way, with continuous calls to next, we are keeping the workers busy downloading data.

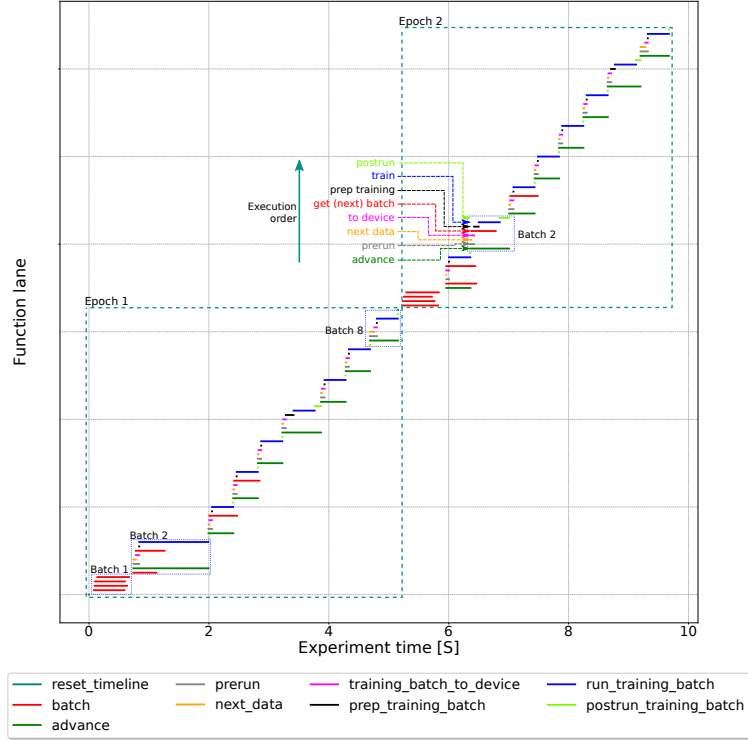


Figure 17: Lightning function execution order, elaborated on a single batch.

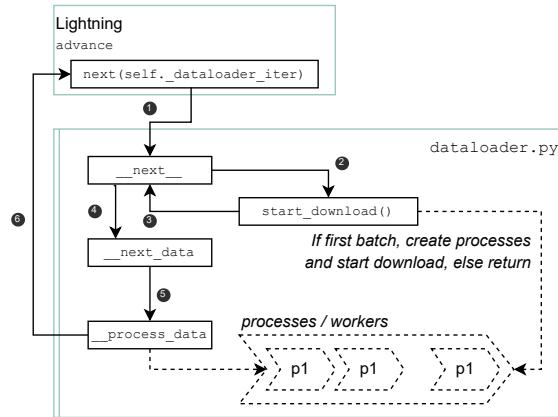


Figure 18: Function calls and the execution order dealing with obtaining the training batch

- *Prep training*, is the lane which shows the runtime of the several functions before the actual training steps, mostly involving hooks and callbacks.
- *Train*, lane showing the training progress
- *Postrun*, lane showing the set of functions, similar to the prep training.

Running multiple test, we noticed that sometimes the prep-training and postrun can take a significant amount of time. Tracing the code, we noticed that the following call in the advance prep-training takes the most:

```
response = self.trainer.call_hook("on_train_batch_start", batch, batch_idx, **extra_kwargs)
```

The longest running code in the call_hook function is:

```
1 callback_fx = getattr(self, hook_name, None)
2 if callable(callback_fx):
3     callback_fx(*args, **kwargs) # <--- slow when "on_train_batch_start"
```

With further tracing, one can get to the `callbacks/base.py`, which is implemented in `callbacks/gpu_stats_monitor.py`. It does nothing but triggers the logger. This indicates that we likely used slightly too aggressive approach to logging. Afterwards, we reduced the logging frequency and (`log_every_n_steps`) removed the `Profiler` from our `Trainer`. This improved the Lightning performance, however, still, compared to raw Torch, in this particular case it was slightly slower. Figure 19 shows the overlap of function timeline calls between Lightning and Torch. It's important to mention there, that the aforementioned logging features (particularly related to GPU) aren't enabled by default, meaning that it should normally not cause any performance issues. We would advise caution while chaining the default settings of Lightning profiler and its logging features.

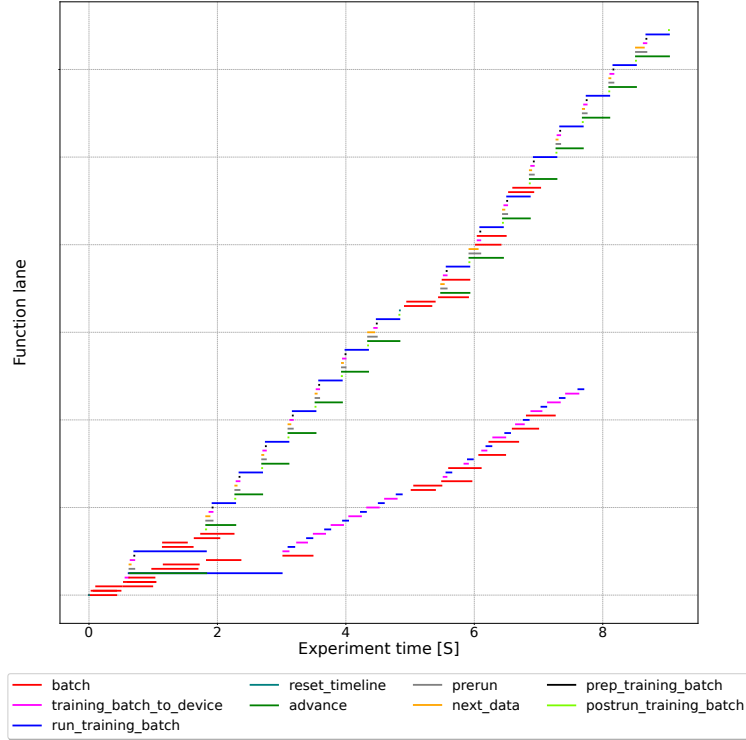


Figure 19: Lightning and Torch function calls overlap.

Pre- and post- training calls, though significantly shorter now, in time build up, making Lightning perform slightly worse than Torch, in this particular example. However this is not a general statement.

A.3.2 Performance differences II: training

Figure 20 shows the throughput and the duration of the training step for Torch and Lightning, with different storage and fetch implementations. The left image shows the throughput in Mbit s^{-1} calculated as $T_{mbits} = \left(DL / \sum_{n=1}^N i_n \right)$ where DL is the size of all the loaded data items (in Mbit s^{-1}) and $i_1, i_2, \dots, i_N$ are the log entries related to the duration of `run_training_batch` function. That said, there are several key points to clarify here:

- *Throughput I*, as expected, at this stage, the data is loaded in memory, and therefore different data loading implementations and storage types should not matter. As seen by the bar plot (orange lines, *Throughput I*), that is the case.
- *Throughput II*, instead of using the function `run_training_batch`, uses `optimizer_step` which runs the training step, and the optimizer step²².
- *Torch vs Lightning*, difference comes from the fact that with Torch implementation, the measurement of GPU throughput is straightforward, as we have a clear and manual access to the forward, backward and optimization step²³, while with Lightning, this is hidden away by the framework generalizations (through

²²https://github.com/PyTorchLightning/pytorch-lightning/blob/1.5.9/pytorch_lightning/loops/optimization/optimizer_loop.py#L246-L269

²³<https://github.com/pytorch/examples/blob/main/imagenet/main.py#L289-L307>

various loops, i.e. epoch, batch, optimization and training loop). In lightning, the advance function of the `training_batch_loop.py`, triggers the advance step of the `training_batch_loop.py`, which after some additional function passing ends up in the `run_optimization` function of the `optimizer_loop.py`. The right side of Figure 20 shows the duration of the aforementioned functions, and for the Lightning the orange training step, is broken down into the training step and the loss update, which takes the majority of the time. This function comes from the automatic optimization, and with our additional tests for using the manual optimization, we got similar numbers. This means that there are also some differences in the implementation of the training step, and that the throughput can only be considered as a wide range between 650 and 3000 Mbit s⁻¹.

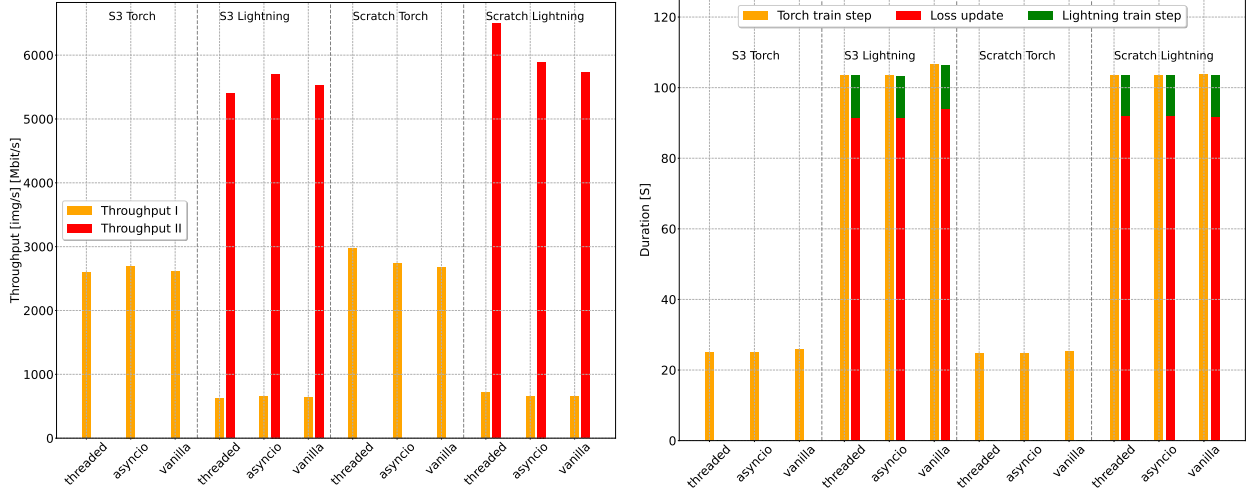


Figure 20: Training phase throughput, for different libraries and implementations (left), along with the cumulative training duration broken down into two longest functions.

Considering the statements above, we can conclude that certain bottlenecks also exist between the data loading process, and the training step itself, which is of scope of this analysis.

A.4 The dreaded GIL

Given that we use AWS S3 as remote object store, we are using the aforementioned Boto3 Python library, and for a sanity check, we performed an experiment, where we use the pure Python multiprocessing and threading, in order to find out whether we are in fact, getting the maximum performance, and to rule out any other factors, not addressed in this work, we might have missed. In addition to this, we made the same experiment using Java. Figure 21 shows the throughput results, for five consecutive experiments, downloading 5000 random images from S3.

The median throughput of 252.18 Mbit s⁻¹ achieved with Python implementation roughly corresponds to the DataLoader performance (shown in Figure 15, as it employs a similar combination of multiprocessing and threading. However, Java implementation achieves 701.39 Mbit s⁻¹ median throughput, which is a significant difference. Also, given the scratch performance shown in Figure 13, with Java threading we would be able to completely replace local storage, with S3.

That said, this indicates that we are *hitting* Python’s concurrency limitations, i.e. the global interpreter lock²⁴ (GIL) prevents *true* parallelism. This, in fact, is what GIL is supposed to do, it ensures that multiple threads are not executing Python bytecode at once and prevents race conditions, thus allowing for thread safety. However, it is not ideal in the sense of taking full advantage of multiprocessing systems. Though there are many pros and cons for GIL, and to Python as the language itself, we will not get into further details here, however, this is an area for why Python was not designed, and we can conclude that the concurrency layer of the data loading should be a part of an external library (e.g. Ray²⁵), written in the lower-level language, like C++, similarly as is the case for lower-level layers of Torch that communicate with the GPU.

²⁴<https://wiki.python.org/moin/GlobalInterpreterLock>

²⁵<https://www.ray.io/>

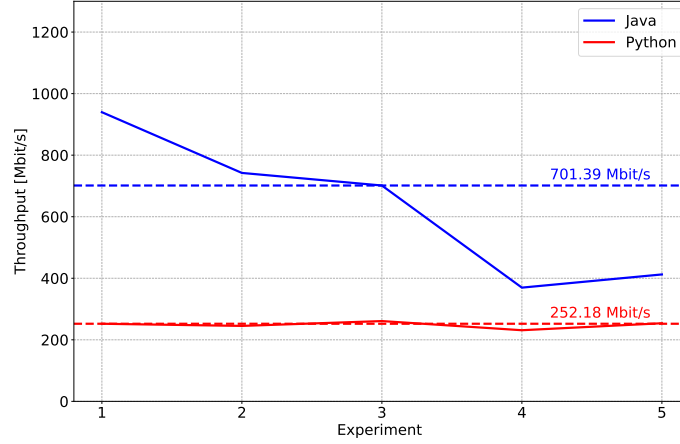


Figure 21: Throughput achieved with S3 for Java and Python, both utilizing concurrency. Dashed lines show the median throughput over 5 experiments.

A.5 Our implementation vs FastAI vs WebDataset

At the moment, there are several ongoing efforts that work on the aforementioned challenge. `WebDataset`²⁶ is an example of a PyTorch Dataset implementation that provides access to data stored in POSIX tar archives. It implements a standard PyTorch `IterableDataset` interface that can use local or remote tar archives, unpack them on the fly, and stream the data items into the `Dataloader`. The `WebDataset` uses a concept called *data shards*, which represent tar files containing a number (or a given size) of data items. During training, the `WebDataset` streams unpacked data items from shards. In the case of using remote storage, it doesn't need to keep the shards locally, as it can sequentially stream its content, with or without cache.

A very similar approach but without streaming is implemented in FastAI [13], which is a DL library developed by the fast.ai research group. In FastAI, data is accessed via `DataBlock`, which is a class that simplifies data management in terms of data preparation and data transforms. This `DataBlock` loads data from a given input path and is afterwards fed into the FastAI data loader. Similar to the data loader implemented in Torch, the number of workers and batch sizes may be given as input. The remote storage can be used through the `untar_data` function, which downloads the entire tar file to a local path. This path may then be used as input to the `DataBlock`.

Figure 22 shows the execution time comparison between the Asyncio dataloader presented in this work (*concurrent*), the FastAI dataloader and WebDataset. In case of WebDataset and FastAI, we use a single shard of 8696 images (986 MB).

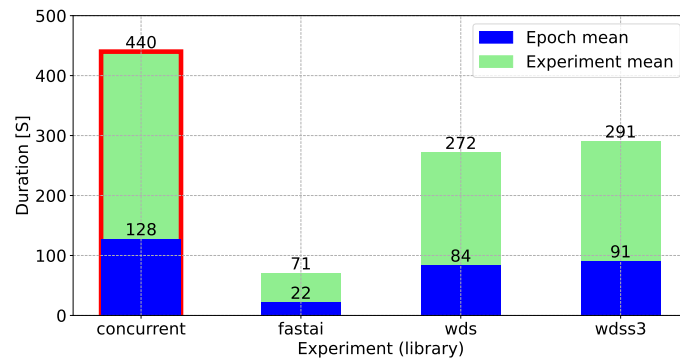


Figure 22: Comparison between our implementation (*concurrent*), with FastAI and WebDataset. Our implementation is highlighted with the red border, and the WebDataset was used with local (*wds*) and a shard stored on AWS S3 (*wdss3*). The image shows the total average runtime of 5 consecutive runs, for the entire experiment and for a single epoch.

²⁶<https://github.com/webdataset/webdataset>

The figure shows the average runtime for 5 consecutive runs. It can be seen that our `ConcurrentDataset` has the highest execution time, while the `FastAI` implementation has the lowest. This substantial differences in runtime may be attributed to the way these data loaders handle the data on the remote storage:

- **Concurrent** For each data item a connection to S3 is established and downloaded before it is used.
- **FastAI** The complete tar archive is downloaded and unpacked and afterwards used by the model.
- **WebDataset** The data are streamed into the model and unpacked on-the-fly.

This indicates that even though our implementation delivers large improvements there are still drawbacks that need to be addressed. This provides a guideline for future work, in which we also need to address networking overheads.

A.6 Fade in and fade out effect

Later on in this work, we perform additional benchmark for different types of storage. Before proceeding, there is an important consideration that needs to be addressed. From the previous experiment, we have chosen a specific example, that best highlights when the image loading function is called, when it finishes and how long it lasts, depending on at which point of the experiment it is called. Figure 23 shows that in the beginning of the experiment we do not have many requests for image loading (fade-in, but as the experiment proceeds the requests slowly build up (fade-out, right histogram)), and similarly, at the beginning of the experiment not many images are still ready (middle histogram). The left most scatter plot shows that at the beginning of the experiment the responses are fast, i.e. the function loading data finishes quickly, however as the experiment proceeds, it reaches a certain peak, after which it, again begins to fall down. So, at the beginning with many requests, the responses are fast, until we saturate the multiprocessing pool, or exhaust the data source.

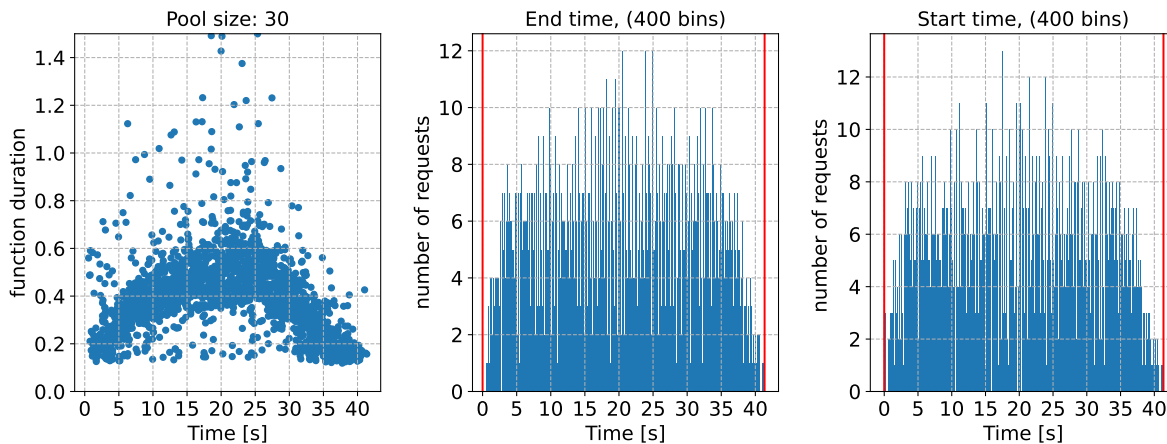


Figure 23: Scatter plot of the `__getitem__` function start time, and its duration throughout a selected experiment (left). Histogram plot showing how many functions `__getitem__` finished (middle) and were started (right) at the certain point of experiment, grouped in 400 bins. The selected experiment uses S3 storage, and is 40.78 s long.

This experiment demonstrates that in order for the experiments to be more convincing, they need to have a long duration, in order for fade-in and fade-out effects to become negligible.